

Day 5: Deposit Amount

1. Day Number

Day 5

2. Topic Name

Arithmetic Operators

Today you will learn how to use arithmetic operators to update a bank balance.

3. Connection with Previous Learning

Previously, you created a bank account and stored its balance.

Today, you will:

1. Ask the user how much money they want to deposit.
2. Add that amount to the existing balance.
3. Display the updated balance.

The basic idea is:

$$\text{Updated balance} = \text{Current balance} + \text{Deposit amount}$$

4. Important Topics

- Numbers
 - Addition operator: +
 - Addition assignment operator: +=
 - Type conversion
 - Updating a balance
 - Functions
 - Basic input validation
-

5. Foundational Notes

Numbers in Python

Python commonly uses two number types:

Integer — `int`

An integer is a whole number.

```
balance = 1000
deposit = 500
```

Float — `float`

A float can contain decimal values.

```
balance = 1000.50
deposit = 250.75
```

For money-related beginner exercises, `float` allows the user to enter both whole numbers and decimal amounts.

The + Operator

The `+` operator adds two numbers.

```
updated_balance = balance + deposit
```

Example:

```
Balance = $1,000
Deposit = $500
Updated balance = $1,500
```

The += Operator

The `+=` operator adds a value to an existing variable and stores the result back in the same variable.

```
balance += deposit
```

This is a shorter version of:

```
balance = balance + deposit
```

Both statements produce the same result.

Type Conversion

The `input()` function always returns text, even when the user enters a number.

For example:

```
deposit_text = input("Enter deposit amount: ")
```

If the user enters 500, Python initially treats it as:

```
"500"
```

This is a string, not a number. Before performing addition, convert it using `float()` or `int()`.

Example:

```
deposit = float(deposit_text)
```

You can also combine input and conversion:

```
deposit = float(input("Enter deposit amount: "))
```

Balance Update

A balance update has three important values:

```
Old balance  
Deposit amount  
New balance
```

The formula is:

```
New balance = Old balance + Deposit amount
```

The original balance variable can then be updated with the new value.

6. Easy Example

Suppose the current balance is:

```
$1,000
```

The user deposits:

```
$250
```

Calculation:

```
$1,000 + $250 = $1,250
```

The updated balance becomes:

```
$1,250
```

A simple arithmetic example in Python is:

```
price = 100  
extra_amount = 20  
  
price += extra_amount  
  
print(price)
```

Output:

```
120
```

This demonstrates how += updates an existing value, but it is not the complete solution to today's problem.

7. Problem Statement

Create a program that:

1. Stores an initial bank balance.
2. Asks the user to enter a deposit amount.
3. Converts the entered value into a number.
4. Checks whether the deposit is valid.
5. Adds the deposit amount to the balance.
6. Displays the updated balance.

Requirement

A negative deposit should not reduce the balance. It should be treated as invalid.

8. Concepts Used

Variables

Variables store the current balance and deposit amount.

```
balance  
deposit_amount
```

User Input

Use `input()` to ask the user for the deposit amount.

Type Conversion

Convert the entered string into a number using `float()`.

Arithmetic Operator

Use `+` or `+=` to add the deposit amount.

Conditional Statement

Use an `if` statement to check whether the deposit amount is negative.

Function

Place the balance-update logic inside a function.

Return Value

The function can return the updated balance to the main part of the program.

9. Thought Process

Before writing code, think through the problem step by step.

Step 1: Create an initial balance

Store the account's current balance in a variable.

```
balance = some starting amount
```

Step 2: Ask for the deposit

Use `input()` to collect the deposit amount from the user.

Step 3: Convert the input

Because `input()` returns a string, convert the value into a number.

Step 4: Validate the deposit

Check whether the deposit amount is negative.

- If it is negative, display an invalid-deposit message.
- If it is zero or positive, continue.

Step 5: Update the balance

Add the deposit amount to the current balance.

```
balance = balance + deposit
```

Or:

```
balance += deposit
```

Step 6: Display the result

Print the updated account balance.

10. Pseudocode

```
START

CREATE a function that accepts current balance and deposit amount

    IF deposit amount is less than zero
        DISPLAY "Invalid deposit amount"
        RETURN the original balance

    ADD deposit amount to current balance

    RETURN updated balance

SET an initial balance

ASK the user to enter a deposit amount

CONVERT the entered value into a number

CALL the deposit function

STORE the returned balance

DISPLAY the updated balance

END
```

11. Suggested Solving Approach: Functional Approach

Use a function to handle the deposit operation.

The function should receive:

```
current balance
deposit amount
```

The function should:

1. Check whether the deposit is valid.
2. Update the balance when valid.
3. Return the updated balance.

Suggested function structure:

```
def deposit_money(current_balance, deposit_amount):
    # Validate the deposit amount

    # Update the balance

    # Return the balance
```

The main program should:

```
Create the starting balance
Take user input
Convert the input
Call the function
Print the returned balance
```

This approach separates the banking logic from user input and output.

12. Easy Edge Cases

Deposit Amount Is 0

Example:

```
Current balance: $1,000
Deposit amount: $0
```

Result:

```
Updated balance: $1,000
```

A zero deposit does not change the balance, but it is not necessarily an error.

Negative Deposit

Example:

```
Current balance: $1,000
Deposit amount: -$200
```

A negative value should not be added because depositing money should not reduce the balance.

Expected behaviour:

```
Invalid deposit amount
Balance remains: $1,000
```

13. Expected Input

Example:

```
Enter deposit amount: 500
```

The initial balance may already be stored in the program, such as:

```
$1,000
```

14. Expected Output

For a valid deposit:

```
Updated balance: $1,500.00
```

For a zero deposit:

```
Updated balance: $1,000.00
```

For a negative deposit:

```
Invalid deposit amount  
Current balance: $1,000.00
```

15. Hint Only

Create a function with two parameters:

```
current balance  
deposit amount
```

Inside the function:

1. Check whether the deposit is less than 0.
2. Return the original balance when it is invalid.
3. Otherwise, use += to update the balance.
4. Return the updated balance.

Remember:

```
deposit_amount = float(input("Enter deposit amount: "))
```

And:

```
balance += deposit_amount
```

Use these ideas to build the complete program yourself.